Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Seminario de Sistemas I.
Ing. Mario Tobar.

“DOCUMENTACIÓN - MANUAL TÉCNICO”

Diego José Maldonado Monterroso.
Carné: 201931811.

Quetzaltenango, Guatemala.

Lunes 26 de Octubre de 2026.

Introducción

El sistema administra la agenda, los cobros y el expediente clínico de una clínica de psicología con una sola profesional. Reemplaza la agenda en papel, la conciliación de transferencias a mano y el expediente disperso. Automatiza la aprobación, los plazos, los recordatorios y los reportes, y deja a las personas solo las decisiones.

Dato	Valor
Producción	`https://pf-clinica.duckdns.org` (panel en `/admin`, portal en `/portal`)
Pruebas	stage en el puerto 8081 y dev en el 8082 del mismo dominio
Código, infraestructura y pipelines	Repositorio público `DiegoMaldonado19/clinica-app` en GitHub
Nube	AWS, región `us-east-1`, creada con Terraform

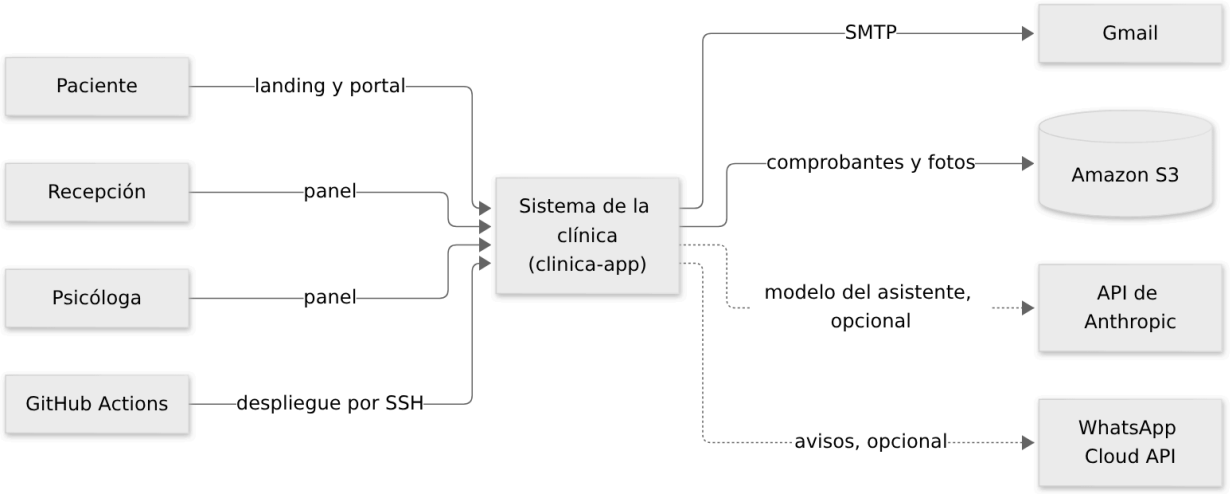
Tres perfiles usan el sistema:

Perfil	Rol en el sistema	Entra por	Hace
Psicóloga	Administradora (`admin`)	Panel `/admin`, con segundo factor obligatorio	Atiende, documenta, asigna pruebas, lee reportes y configura la clínica
Recepción	Personal (`secretary`)	Panel `/admin`, con segundo factor obligatorio	Aprueba solicitudes, concilia pagos, agenda por teléfono y registra llegadas
Paciente	Usuario (`patient`)	Landing y portal `/portal`	Agenda, paga, cancela o reprograma y responde las pruebas asignadas

Este manual describe la arquitectura detallada, el modelo de datos, la guía de despliegue y la trazabilidad del sistema tal como quedó construido y desplegado el 25 de octubre de 2026.

Arquitectura

Contexto



El sistema es un monolito modular en Laravel con un servicio aparte en Python para el asistente conversacional: es el backend heterogéneo. Laravel atiende la landing, el portal, el panel y una API interna; el asistente consulta esa API y nunca la base de datos.

Tecnologías

Capa	Tecnología y versión
Aplicación	PHP 8.4.26, Laravel 13.31, Filament 5.8.4, Livewire 4.4.8, Alpine, Tailwind 4 y Vite 8
Reportes	laravel-dompdf (PDF) y openspout (Excel)
Asistente	Python 3.13.16, FastAPI 0.142.4, SDK `anthropic` 1.12.1 y Claude Haiku 5.5
Datos	MariaDB 11.4.13 (RDS) y Redis 7.4.11 (solo caché)
Calidad	Pest 5, PHPStan con Larastan, Deptrac 4 y Pint; ruff, mypy, pytest y pip-audit en el asistente; `composer audit` y gitleaks
Contenedores	Docker 29.9.1, Compose 5.6.0 y nginx 1.30.5 sin privilegios
Nube	AWS: EC2, RDS, S3, AWS Backup, IAM con OIDC y Budgets; Terraform 1.16.6
Entrega	GitHub Actions con Environments; imágenes en Docker Hub

Interfaces

Interfaz	Ruta	Tecnología	Quién la usa
Landing	`/`	Livewire y Alpine	Público: servicios, preguntas frecuentes, agendamiento y asistente
Agendamiento	`/agendar`	Livewire (`BookingWizard`)	Pacientes nuevos o recurrentes
Portal	`/portal`	Panel de Filament `portal`	Pacientes: citas, pagos, pruebas y perfil
Panel	`/admin`	Panel de Filament `admin`, con segundo factor obligatorio	Recepción y la psicóloga
API del asistente	`/api/v1`	Laravel con token de servicio, contrato OpenAPI 3.1 en `docs/api/openapi.yaml`	Solo el contenedor del asistente
Salud	`/up`	Laravel	El pipeline y el chequeo de salud de nginx

Método y ruta de la API	Qué hace
`GET /api/v1/knowledge`	Devuelve la base de conocimiento: preguntas frecuentes, servicios y tarifas vigentes
`GET /api/v1/availability`	Horarios libres de un servicio en un día, con las reglas de anticipación aplicadas
`POST /api/v1/appointments`	Solicita una cita con origen «chat»; pasa por las mismas reglas que la landing
`POST /api/v1/crisis-alerts`	Avisa con urgencia a la psicóloga (NT-24) sin datos de la persona

Arquitectura hexagonal

El dominio no conoce Laravel. Las reglas de negocio viven en `src/<Contexto>/Domain/` y no importan `Illuminate\*`. Laravel, Filament y la base de datos son adaptadores que se conectan desde afuera.



Deptrac verifica la regla de dependencia en cada Pull Request (`deptrac --fail-on-uncovered`): una dependencia al revés rompe la construcción. Las pruebas unitarias del dominio corren sin base de datos ni framework.

Contextos acotados

Contexto	Responsabilidad	Piezas principales
Scheduling	Disponibilidad, pre-citas, aprobación, SLA, cancelación, reprogramación, inasistencia y bloqueos	Agregado `Appointment` con su máquina de 13 estados, `BookingService`, `AppointmentTransitions`, `TieredCancellationPolicy` y `AvailabilityCalculator`
Billing	Pagos, comprobantes, conciliación, cargos y crédito a favor	`Payment`, `CreditAllocation`, `BillingService` y los puertos `FiscalDocumentIssuer` y `PaymentGateway`
ClinicalRecords	Expediente, notas SOAP cifradas e inmutables, pruebas PHQ-9 y GAD-7	`ClinicalNote`, `PsychTest`, `PsychTestAssignment`, `PsychTestService` y `ClinicalCipher`
Identity	Pacientes y su registro	`PatientRegistry`; usuarios, roles y segundo factor los resuelve Filament con las políticas de `app/Policies`
Notifications	Plantillas, canales, preferencias, ventana de silencio, idempotencia y reintentos	`NotificationDispatcher`, `SendNotificationJob`, `MailChannel` y `WhatsAppChannel`
Analytics	Los 13 indicadores y los reportes PDF y Excel	`ReportService`, `ReportPeriod`, `Report` y `Kpi`
Shared	Reloj, bus de eventos, reglas de negocio y objetos de valor	`ClockInterface`, `EventBus`, `BusinessRules` y `Money`
Asistente (servicio aparte)	Conversación, agendamiento por la API y crisis	`chatbot/app`: `crisis.py` (clasificador determinista), `assistant.py` y `laravel.py`

Comunicación entre contextos

Los contextos se comunican solo por eventos de dominio, sin uniones SQL entre ellos.

Emisor	Eventos
Scheduling	`PreAppointmentRequested`, `AppointmentApproved`, `AppointmentRejected`, `PreAppointmentExpired`, `PayAtDeskChosen`, `AppointmentFullyBooked`, `AppointmentCheckedIn`, `AppointmentCancelled`, `AppointmentCancelledUnpaid`, `AppointmentNoShow` y `ReminderDue`

Emisor	Eventos
Billing	`PaymentProofSubmitted`, `PaymentApproved`, `PaymentRejected`, `PaymentSettledByCredit`, `LateCancellationFeeApplied`, `LateCancellationFeeWaived` y `PatientCreditIssued`
ClinicalRecords	`ClinicalNoteSealed`, `PsychTestAssigned`, `PsychTestSubmitted` y `PsychTestOverdue`
Identity	`PatientAccountCreated`

Suscriptor	Reacciona
`SchedulingEventSubscriber`	Al pedir o aprobar una cita programa sus trabajos diferidos (expiración, impago y recordatorios), y responde a los pagos y a la nota sellada
`BillingEventSubscriber`	Abre o liquida el cobro ante la aprobación, el pago en caja, la llegada y las cancelaciones
`DomainEventNotifier`	Convierte cada evento en su aviso

Puertos y adaptadores

Cada puerto se une a su adaptador en `AppServiceProvider`: cambiar un adaptador es una línea.

Puerto	Adaptador actual	Alternativa prevista
`AppointmentRepository`, `AvailabilityProvider`, `ServiceCatalog`	`Database…` sobre MariaDB	—
`SlotLockManager`	Candado de caché, con el índice único `(therapist_id, starts_at, active_slot)` como red de seguridad	Bloqueo pesimista
`PatientRegistry`, `PaymentRepository`	`Database…`	—
`FiscalDocumentIssuer`	`NullFiscalDocumentIssuer`	Un certificador de facturas FEL
`PaymentGateway`	`NullPaymentGateway`	Una pasarela de tarjeta
`ClinicalNoteRepository`, `PsychTestAssignmentRepository`	`Database…`, con `ClinicalCipher`	—
`NotificationChannel`	`MailChannel` y, si se configura, `WhatsAppChannel`	SMS
`MetricsSource`	`DatabaseMetricsSource`	Tablas resumen
`ReportExporter`	`ReportFileExporter` (openspout y dompdf)	—

Puerto	Adaptador actual	Alternativa prevista
`ClockInterface`	`SystemClock` (`FrozenClock` en pruebas)	—
`EventBus`	`LaravelEventBus`	Un broker de mensajes
`BusinessRules`	`BusinessRuleSettings`: RN-01 a RN-20 en la base, con vigencia	—

Estructura del repositorio

```
clinica-app/
├── src/<Contexto>/{Domain,Application,Infrastructure}   dominio hexagonal
(Deptrac)
├── app/                 adaptadores de Laravel: Filament (panel y portal),
Livewire (landing y widget),
│                        API, modelos de lectura, políticas y listeners
├── database/            18 migraciones y semillas (catálogos, reglas,
permisos, plantillas)
├── chatbot/             asistente en Python: FastAPI, clasificador de crisis y
sus pruebas
├── docker/              Dockerfile con destinos dev, prod y web; configuración
de nginx
├── infra/               Terraform de AWS y su bootstrap
├── scripts/             deploy.sh y tls.sh, que corren en la EC2
├── .github/workflows/   ci-cd.yml (calidad y despliegue) e infra.yml
(Terraform con aprobación)
├── tests/               Unit (dominio sin base), Integration y Feature
└── docs/                contrato OpenAPI, decisiones de la fase 3, runbook y
documentación de la fase 2
```

Seguridad

Capa	Cómo se protege
Contraseñas	Hash con bcrypt. Al menos 12 caracteres y no filtradas en brechas conocidas. La temporal vence en 72 horas y obliga a cambiarla
Segundo factor	Código de 6 dígitos por correo (vence en 10 minutos) o aplicación TOTP con 8 códigos de recuperación. Obligatorio para el personal, opcional para pacientes
Recuperación de contraseña	Enlace firmado de un solo uso que vence en 60 minutos, con una solicitud por minuto. La respuesta es igual exista o no la cuenta, y al cambiar la contraseña se cierran las demás sesiones
Sesiones	En la base de datos, con cookies `secure`, `httponly` y `samesite`, regeneradas al ingresar
Autorización	32 habilidades asignadas por rol y políticas por recurso. El paciente entra solo al portal y solo ve lo suyo
Datos clínicos	Notas SOAP y respuestas de pruebas cifradas por la aplicación con `CLINICAL_ENCRYPTION_KEY`, distinta de `APP_KEY`. *Triggers* impiden cambiar lo sellado o respondido y borrar cualquier registro clínico

Capa	Cómo se protege
Bitácora	`audit_log` sin `UPDATE` ni `DELETE` posibles. Registra cada lectura del expediente y de un resultado, cada intento denegado y cada código de segundo factor equivocado
Archivos	Comprobantes y fotos en S3 privado, servidos por la aplicación con permiso: 404 a quien no puede verlos. El comprobante se valida como PDF por su contenido
Transporte	HTTPS con HSTS y cabeceras de seguridad, redirección de HTTP, sin contenido mixto y TLS obligatorio hacia RDS
Reposo	Discos de la EC2 y de RDS cifrados; buckets privados con versionado
Plataforma	Contenedores sin root, IMDSv2 obligatorio y tres roles IAM de mínimo privilegio; sin usar la cuenta raíz
Secretos	Environments de GitHub, escritos en el `.env` de la EC2 por el pipeline. Ninguno en el repositorio; gitleaks revisa cada Pull Request
Asistente	Ningún dato de pacientes llega al modelo. Un clasificador determinista detecta la crisis antes del modelo, con una batería de 20 casos en la CI

Área	Habilidades	Psicóloga	Recepción	Paciente
Citas de todos	Ver, crear, aprobar, rechazar, cancelar, registrar llegada e inasistencia	Sí	Sí	No
Citas propias	Ver, solicitar y cancelar las suyas	Sí	Sí	Sí
Agenda	Bloquear horarios	Sí	No	No
Pagos de todos	Ver, registrar y aprobar	Sí	Sí	No
Pagos propios	Ver y registrar los suyos	Sí	Sí	Sí
Cargos	Exonerar un cargo por cancelación	Sí	No	No
Pacientes	Ver, crear y editar la ficha y el motivo de consulta	Sí	Sí	No
Expediente clínico	Ver, crear, sellar y enmendar notas; asignar pruebas	Sí	No	No
Pruebas	Responder las asignadas	No	No	Sí
Personal	Crear usuarios y roles	Sí	No	No
Reportes	Indicadores operativos / con ingresos	Sí / Sí	Sí / No	No
Configuración	Reglas, servicios, preguntas frecuentes y bitácora	Sí	No	No

La administradora puede crear roles nuevos para el personal con las habilidades que elija, salvo las clínicas, que siguen siendo exclusivas de la psicóloga.

Integraciones

Integración	Cómo	Estado
Correo	SMTP de Gmail (587) con contraseña de aplicación, por la cola `mail`	Activo: segundo factor, recuperación y avisos
Almacenamiento	Amazon S3 con el rol IAM de la instancia	Activo: comprobantes y fotos de perfil
Asistente	Contenedor FastAPI que habla con Laravel por `/api/v1` con token	Desplegado; se activa con la llave del modelo
WhatsApp	Canal `WhatsAppChannel`, con una plantilla aprobada por Meta	Listo; se activa con dos secretos
Facturación FEL y pasarela de tarjeta	Puertos con adaptador nulo	Listos para un adaptador cuando haya contrato
Certificados	Let's Encrypt por DNS en DuckDNS, renovado a diario	Activo

Reportes y métricas

El tablero muestra 13 indicadores calculados al vuelo para el periodo y el servicio elegidos. El periodo puede ser este mes, el mes anterior, los últimos 90 días, este año o un rango elegido de hasta un año. Los widgets y las dos exportaciones salen del mismo cálculo, así que los totales no pueden diferir. Recepción recibe el mismo reporte sin ninguna cifra de ingresos.

Indicador	Cálculo
KPI-01 Inasistencia	Inasistencias entre las citas que llegaron a agendarse
KPI-02 Tiempo de aprobación	Promedio entre la solicitud y la aprobación, solo en las solicitudes en línea
KPI-03 Solicitudes expiradas	Expiradas entre las solicitudes en línea del periodo
KPI-04 Conversión de la landing	Sin dato: no se miden visitantes
KPI-05 Tiempo de conciliación	Promedio entre el comprobante subido y su aprobación
KPI-06 Ocupación de agenda	Horarios ocupados entre horarios publicados, menos los bloqueos
KPI-07 Ingreso cobrado	Pagos aprobados por transferencia o efectivo, por fecha de aprobación
KPI-08 Cancelación tardía	Cancelaciones con recargo entre las citas aprobadas
KPI-09 Adherencia	Sesiones atendidas entre las que llegaron a agendarse
KPI-10 Entrega de notificaciones	Enviadas entre enviadas y fallidas
KPI-11 Sin intervención humana	Citas solicitadas por la web o el asistente entre todas
KPI-12 Notas en 24 horas	Notas selladas a tiempo entre las sesiones cuyo plazo venció
KPI-13 Ocupación sobre capacidad	Sesiones atendidas entre la capacidad nominal de los días transcurridos

Exportación. «Exportar Excel» genera un .xlsx con una hoja de indicadores y otra por gráfica (openspout). «Exportar PDF» genera el reporte con el periodo, los indicadores y las tablas (dompdf). Ambas se descargan al momento.

Calidad y pruebas

Verificación	Herramienta	Cuándo
Formato PSR-12	Pint	Cada Pull Request
Análisis estático	PHPStan con Larastan	Cada Pull Request
Regla de dependencia	Deptrac, sin violaciones ni clases sin clasificar	Cada Pull Request
Pruebas de PHP	Pest: 318 pruebas unitarias, de integración y funcionales sobre MariaDB, más 199 en una corrida sin caché	Cada Pull Request
Pruebas del asistente	pytest, con la batería de 20 casos de crisis al 100 %, más ruff y mypy	Cada Pull Request
Dependencias vulnerables	`composer audit` y pip-audit	Cada Pull Request
Secretos en el código	gitleaks	Cada Pull Request
Salud desplegada	`/up` desde fuera de la EC2	Cada despliegue

Decisiones de arquitectura

ADR	Decisión	Estado
001	Laravel 13 sobre PHP 8.4	Vigente
002	Arquitectura hexagonal con el dominio aislado	Vigente, verificada por Deptrac
003	Monolito modular	Vigente
004	Livewire y Alpine para el frente público	Vigente
005	Filament 5 para el panel	Vigente
006	Base de datos gestionada	Vigente (RDS)
007	Sin multicliente por ahora	Vigente
008	Redis solo como caché; sesiones y colas en la base	Vigente
009	El asistente es un servicio aparte en Python	Vigente
010	Claude Sonnet 5	Reemplazada por la 030
011	Recuperación estática de contexto	Vigente
012	El asistente consume la API, nunca la base	Vigente
013	Balanceador y dos instancias	Diferida por la 023
014	Respaldos con AWS Backup	Vigente
015	Despliegue por gestión remota sin SSH	Reemplazada por la 027

ADR	Decisión	Estado
016	Registro privado ECR	Reemplazada por la 025
017	Solo correo en la primera fase	Vigente, con Gmail (029)
018	Segundo factor por código de correo	Vigente, con el MFA de Filament (031)
019	UUID v7 como clave primaria	Vigente
020	Terraform para toda la infraestructura	Vigente
021	Desarrollo basado en tronco con `main` única	Vigente
022	EC2 con Docker Compose, no ECS	Vigente
023	Una EC2 para los tres entornos	Vigente
024	Redis en contenedor	Vigente
025	Docker Hub con promoción por árbol de archivos	Vigente
026	Secretos en Environments de GitHub	Vigente
027	Despliegue por SSH	Vigente
028	Let's Encrypt por DNS en DuckDNS	Vigente
029	Correo por Gmail	Vigente
030	Claude Haiku 5.5 para el asistente	Vigente
031	Segundo factor de Filament: correo o aplicación	Vigente
032	Reportes al vuelo, sin tablas resumen	Vigente
033	Token de servicio entre Laravel y el asistente	Vigente
034	El asistente agenda por formulario	Vigente
035	Canales de aviso activos por configuración	Vigente
036	Una plantilla genérica de WhatsApp	Vigente
037	Puertos de facturación y pasarela con adaptador nulo	Vigente
038	El asistente se activa con su llave	Vigente
039	La sesión se cierra sola al terminar	Vigente
040	Roles del personal creados desde el panel	Vigente
041	Fotos de perfil servidas por la aplicación	Vigente

Modelo de datos

Reglas del esquema

Regla	Cómo se cumple
Claves primarias	UUID v7 (`CHAR(36)`) en las entidades de dominio; `BIGINT` autoincremental en catálogos y bitácoras
Dinero	`INT` de centavos más el código de moneda. Nunca `FLOAT`
Fechas	`DATETIME` en UTC. La hora de Guatemala solo al presentar
Claves foráneas	`ON DELETE RESTRICT`: nada clínico ni financiero se borra en cascada
Multicliente	No existe `tenant_id`: hay una sola clínica
Contenido clínico	Cifrado en la aplicación; la base solo ve texto cifrado
Inmutabilidad	*Triggers* en `clinical_notes`, `psych_test_assignments` y `audit_log`
Contextos	Cada tabla pertenece a un contexto y no hay uniones SQL entre contextos; los reportes leen cada contexto por separado
Charset	`utf8mb4_unicode_ci` en toda tabla

Identidad y acceso

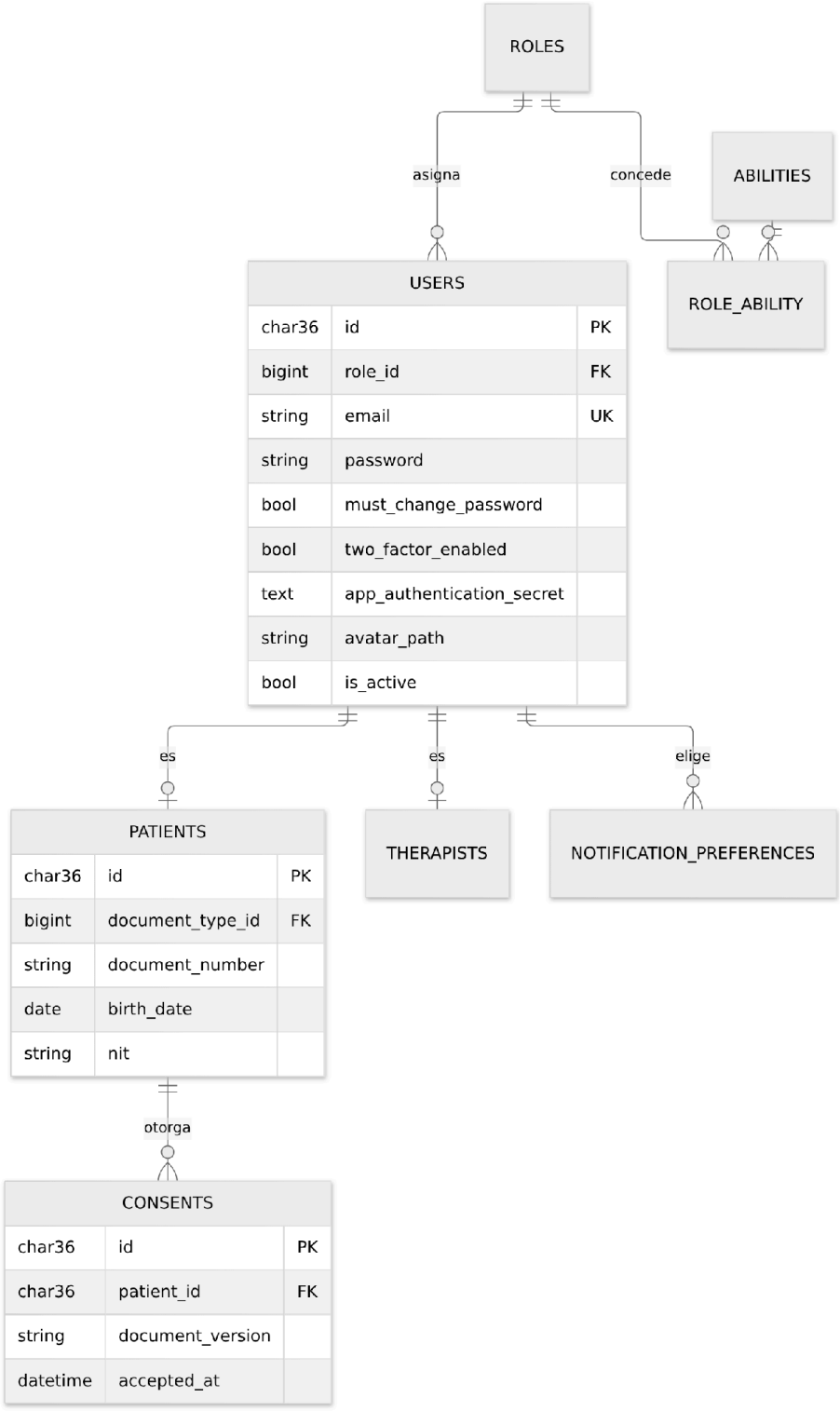


Tabla	Guarda	Claves y defensas
`users`	Personas: rol, nombre, correo, contraseña con hash, credencial temporal, segundo factor (cifrado), foto e `is_active`	`email` único; nunca se borran, se desactivan
`roles`, `abilities`, `role_ability`	Tres roles sembrados más los que crea la administradora; 32 habilidades	Clave compuesta en el pivote
`patients`	Ficha: documento, sexo, nacimiento, contacto de emergencia y NIT. Su `id` es el del usuario	`(document_type_id, document_number)` único
`therapists`	La psicóloga; su `id` es el del usuario	El sistema supone una sola fila
`consents`	Cada aceptación de la política: documento, versión, fecha e IP	—

Agenda

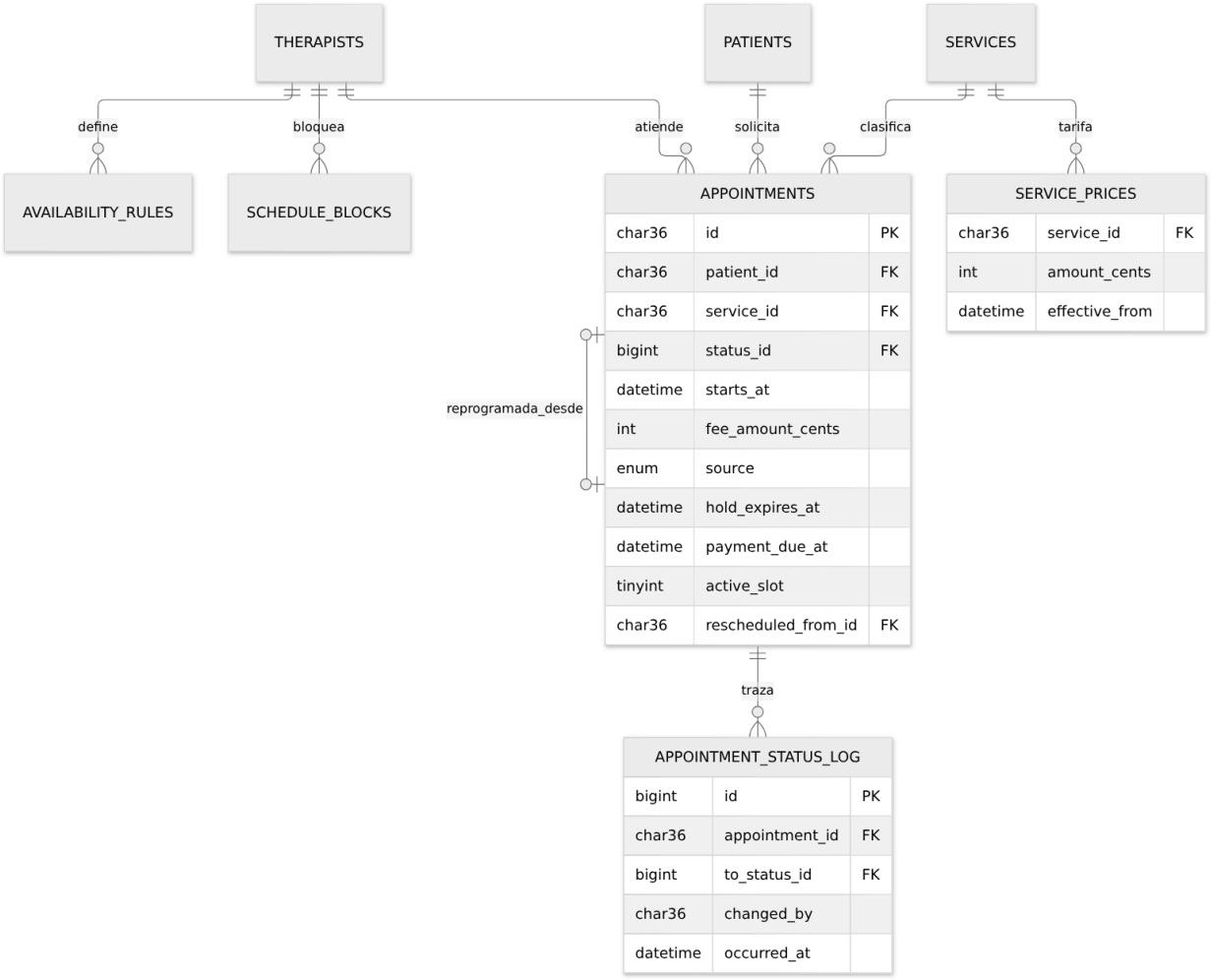


Tabla	Guarda	Claves y defensas
`services`, `service_prices`	Servicios y tarifas en centavos con fecha de vigencia	`(service_id, effective_from)` único
`availability_rules`	El horario semanal de la psicóloga	—
`schedule_blocks`	Vacaciones y ausencias	—
`appointments`	La cita: estado, horario, tarifa congelada, origen (`web`, `chat`, `phone` o `walk_in`), plazos de apartado y de pago, aprobación, llegada y cita de origen si se reprogramó	`(therapist_id, starts_at, active_slot)` único: impide la doble reserva. `active_slot` vale 1 mientras la cita ocupa el horario
`appointment_status_log`	Cada transición, con quién y por qué; `changed_by` nulo es el sistema	Solo se inserta

Cobros

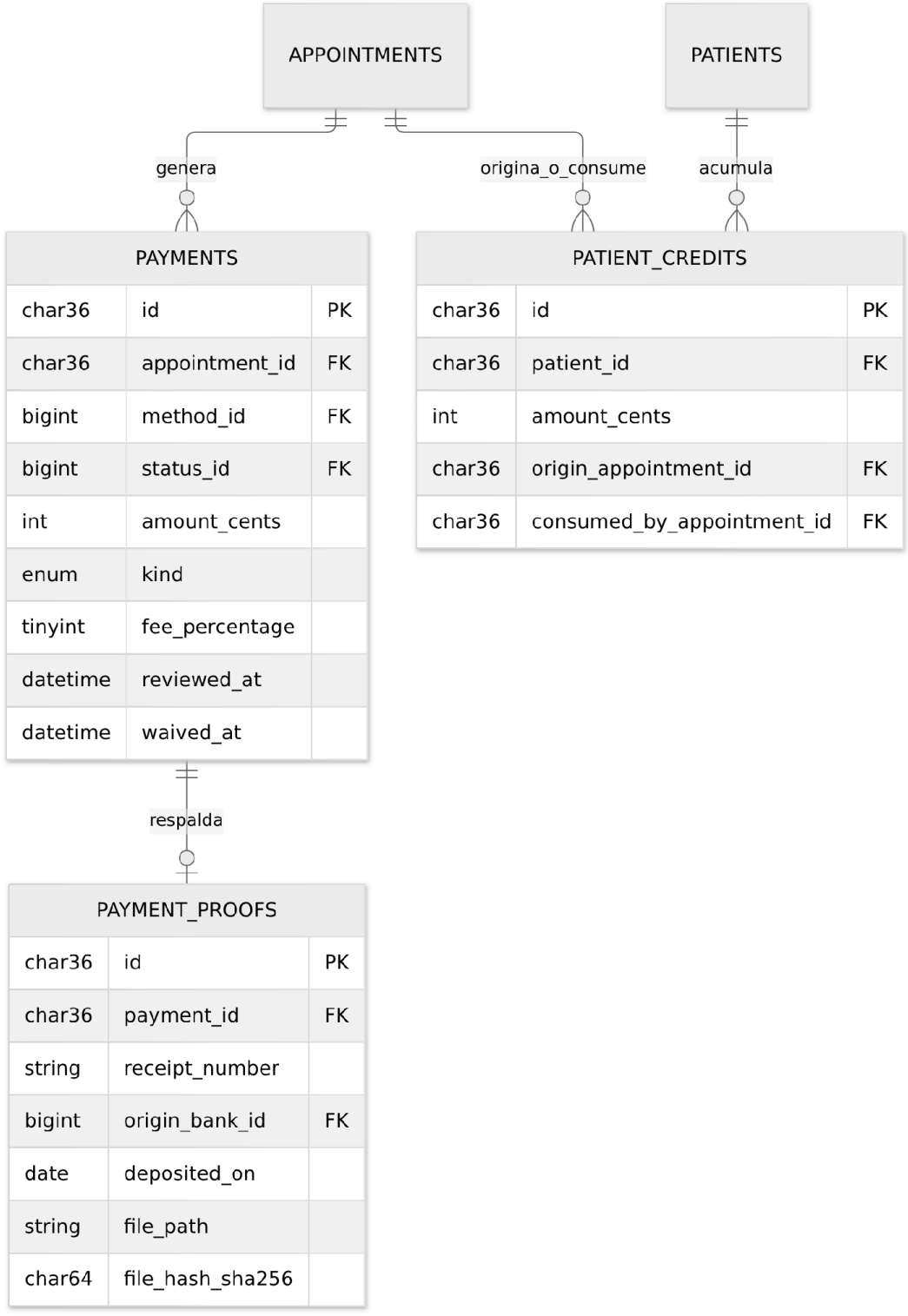

APPOINTMENTS
PATIENTS
genera
origina_o_consume
acumula
PAYMENTS
char36 id PK
char36 appointment_id FK
bigint method_id FK
bigint status_id FK
int amount_cents
enum kind
tinyint fee_percentage
datetime reviewed_at
datetime waived_at
PATIENT_CREDITS
char36 id PK
char36 patient_id FK
int amount_cents
char36 origin_appointment_id FK
char36 consumed_by_appointment_id FK
respalda
PAYMENT_PROOFS
char36 id PK
char36 payment_id FK
string receipt_number
bigint origin_bank_id FK
date deposited_on
string file_path
char64 file_hash_sha256

Tabla	Guarda	Claves y defensas
`payments`	Cobro de la sesión o cargo por cancelación: método, estado, monto, porcentaje, revisión y exoneración	—
`payment_proofs`	El comprobante: boleta, banco, últimos 4 dígitos, fecha de depósito, ruta del PDF en S3 y su huella	`(origin_bank_id, receipt_number)` único: una boleta no se usa dos veces
`patient_credits`	Crédito a favor: de qué cita salió y en cuál se consumió	Se consume del más antiguo al más nuevo

Expediente clínico

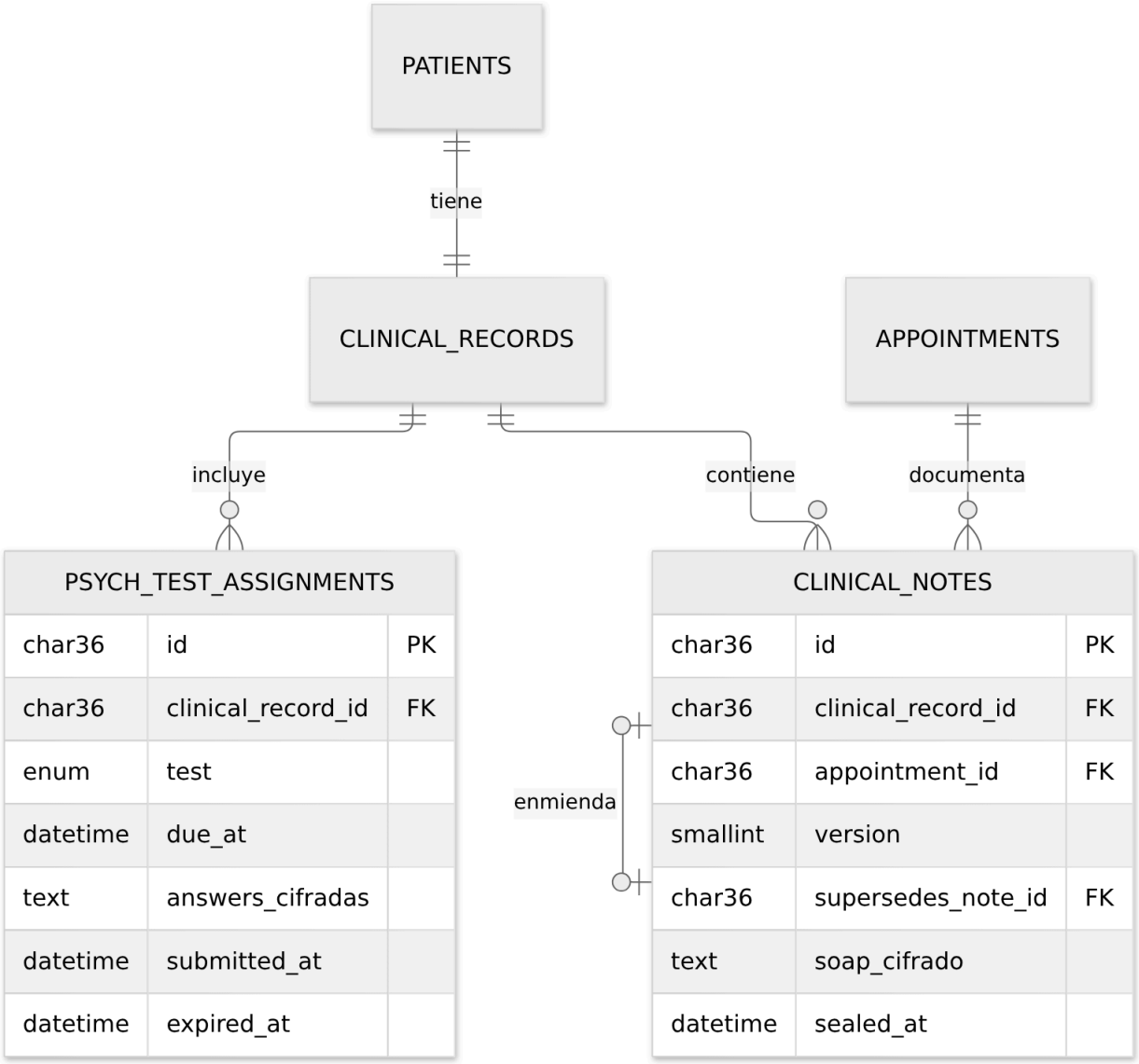


Tabla	Guarda	Claves y defensas
clinical_records	Ficha administrativa: motivo de consulta y quién refiere. Recepción la ve	patient_id único
clinical_notes	Nota SOAP cifrada en cuatro columnas, versión, enmienda y sellado	(appointment_id, version) único; *triggers*: lo sellado no cambia y nada se borra
psych_test_assignments	PHQ-9 o GAD-7, fecha límite, respuestas cifradas, respondida o vencida. El resultado se calcula, no se guarda	*Triggers*: lo respondido no cambia y nada se borra

Avisos y tablas transversales

Tabla	Guarda	Claves y defensas
notification_templates	Plantillas NT-01 a NT-25: asunto y cuerpo	code único
notification_dispatches	Cada envío: plantilla, canal, destinatario, estado, intentos, hora programada y último error	idempotency_key único: el mismo aviso no sale dos veces
notification_preferences	Canales que eligió cada paciente	(user_id, channel_id)
business_rule_settings	Parámetros de RN-01 a RN-20 con historial por vigencia	(rule_code, param_key, effective_from)
audit_log	Quién hizo qué, sobre qué, cuándo y desde qué IP	*Triggers*: sin UPDATE ni DELETE
faqs	Preguntas frecuentes de la landing y del asistente	—
Catálogos (12)	Roles, habilidades, estados de cita y de pago, métodos, bancos, canales, estados de aviso, motivos de cancelación, categorías, sexos y tipos de documento	code único; la semilla los reescribe en cada despliegue
Del framework	sessions, jobs, job_batches, failed_jobs, cache, cache_locks y password_reset_tokens	Sesiones y colas en la base

Reglas de negocio

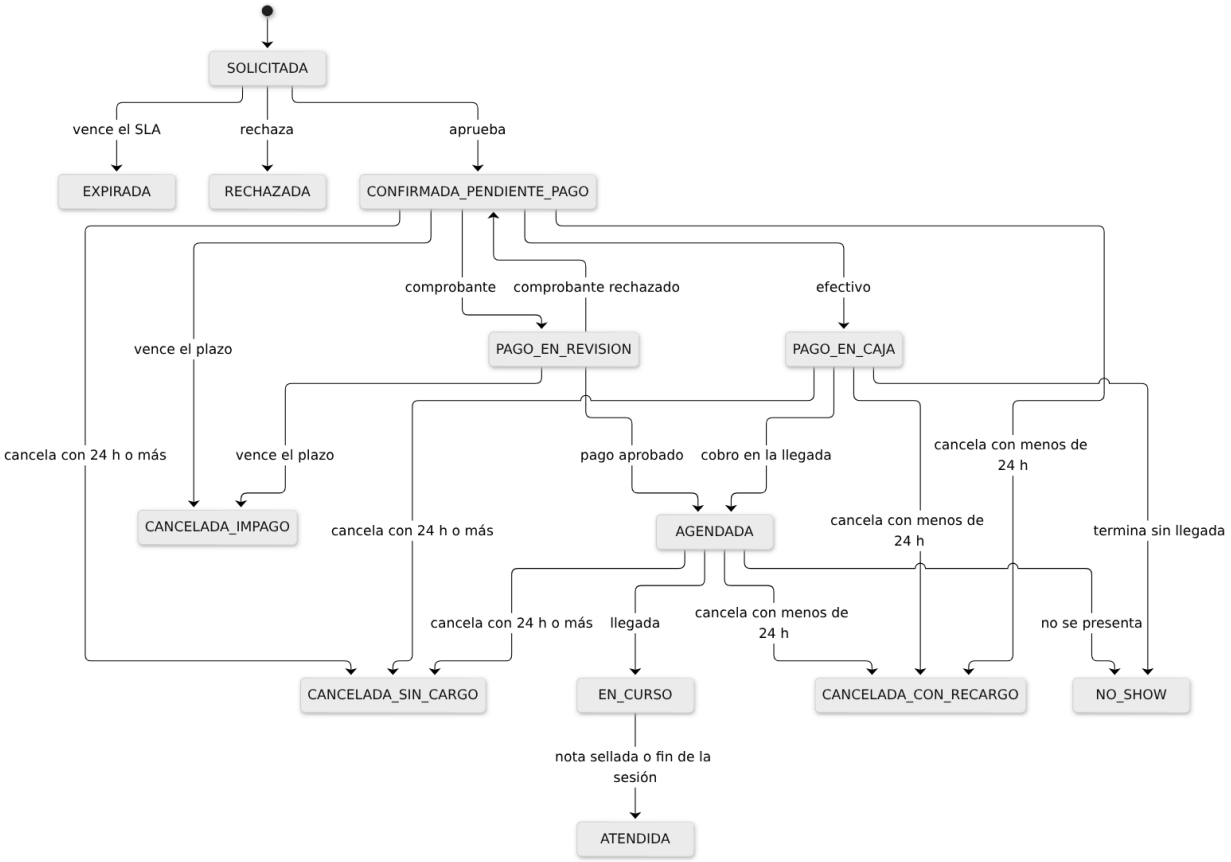
Los procesos aplican estas reglas. Las que tienen parámetro se cambian desde el panel, con fecha de vigencia, sin desplegar.

Regla	Qué exige	Parámetro y valor
RN-01	Sin agendamiento en línea para el mismo día; se ofrece el teléfono	same_day_booking_enabled = no
RN-02	Anticipación mínima para agendar en línea	24 h

Regla	Qué exige	Parámetro y valor
RN-03	La pre-cita se resuelve dentro del SLA o expira	24 h hábiles, 72 h en fin de semana
RN-04	El horario queda apartado durante el SLA	Derivado de RN-03
RN-05	El pago debe estar aprobado antes de la sesión	3 h antes
RN-06	Cancelación escalonada	24 h o más: sin cargo; de 24 a 1 h: 50 %; menos de 1 h o no asistir: 100 %
RN-07	Una cita solo queda agendada con el pago aprobado o cobrado	—
RN-08	Recordatorios automáticos	24 h y 2 h antes
RN-09	Todo pago lo aprueba una persona	Aprobación automática = no
RN-10	Solo la psicóloga edita el contenido clínico	—
RN-11	Una nota sellada no cambia; la corrección es una versión nueva	—
RN-12	Todo acceso al expediente queda en la bitácora	—
RN-13	Cada paciente ve solo sus datos	—
RN-14	Una boleta no se repite dentro del mismo banco	—
RN-15	Contraseña temporal con vencimiento y código de segundo factor	72 h y 10 min
RN-16	La cancelación sin cargo de una cita pagada deja crédito a favor	Crédito, no reembolso
RN-17	Avisos idempotentes y con ventana de silencio	21:00 a 07:00, salvo urgentes
RN-18	Bloquear la agenda obliga a resolver cada cita afectada	—
RN-19	El asistente no toca la base: usa la API con permisos acotados	—
RN-20	Ante señales de crisis: línea de emergencia, alerta y cierre de la conversación	Activo

Máquina de estados de la cita

Ninguna cita cambia de estado fuera de este diagrama: 13 estados y 20 transiciones, definidos en el dominio (`AppointmentStatus`).



Liberan el horario RECHAZADA, EXPIRADA y las tres cancelaciones. La inasistencia no lo libera: la hora ya se consumió. La conciliación de cada 5 minutos aplica sola los vencimientos y cierra las sesiones terminadas.

Migraciones

Son versionadas, corren solas en cada despliegue (`migrate --force`) y siempre hacia adelante.

Etapa	Migración	Qué crea
Base	`0001_01_01_000000…000002`	Usuarios, sesiones, caché y colas
Fase 2	`2026_09_12_100000_create_catalog_tables`	Los 12 catálogos
Fase 2	`2026_09_12_100100_create_business_rule_settings_table`	Reglas de negocio con vigencia
Fase 2	`2026_09_12_200000_create_identity_tables`	Pacientes, psicóloga y permisos
Fase 2	`2026_09_18_100000_create_notification_tables`	Plantillas y envíos

Etapa	Migración	Qué crea
Fase 2	`2026_09_18_200000_create_scheduling_tables`	Servicios, tarifas, horario, citas y su bitácora
Fase 2	`2026_09_18_300000_create_booking_tables`	Consentimientos y NIT
Fase 2	`2026_09_18_400000_create_billing_tables`	Pagos, comprobantes y créditos
Fase 2	`2026_09_18_500000_create_clinical_tables`	Expediente y notas con sus *triggers*
Fase 2	`2026_09_18_600000_create_audit_and_block_tables`	Bitácora inmutable y bloqueos
Fase 3	`2026_10_19_100000_add_app_authentication_to_users_table`	Segundo factor por aplicación
Fase 3	`2026_10_21_100000_create_faqs_table`	Preguntas frecuentes editables
Fase 3	`2026_10_22_100000_add_rescheduled_from_to_appointments`	Cita de origen de una reprogramada
Fase 3	`2026_10_22_100100_create_notification_preferences_table`	Canales elegidos
Fase 3	`2026_10_23_100000_create_psych_test_assignments_table`	Pruebas psicométricas con sus *triggers*
Fase 3	`2026_10_25_100000_add_avatar_path_to_users_table`	Foto de perfil

Diferencias con el diseño

Diseño	Construido	Por qué
Tabla `slot_holds`	El apartado vive en la cita (`active_slot` y `hold_expires_at`)	Un índice único sobre la cita basta
Tabla `two_factor_codes`	No existe	El código se guarda con hash en la sesión
Tabla `reminders`	No existe	Los recordatorios son trabajos diferidos con llave de idempotencia
Tres tablas de pruebas	Una sola, `psych_test_assignments`	Los instrumentos son un enum del dominio y el resultado se calcula
Tablas de conversaciones	No existen	El asistente no guarda conversaciones
Tablas resumen	No existen	Reportes al vuelo, suficientes a este volumen
Pago en caja → cancelación solamente	También pago en caja → inasistencia	La sesión sin llegada se cierra sola

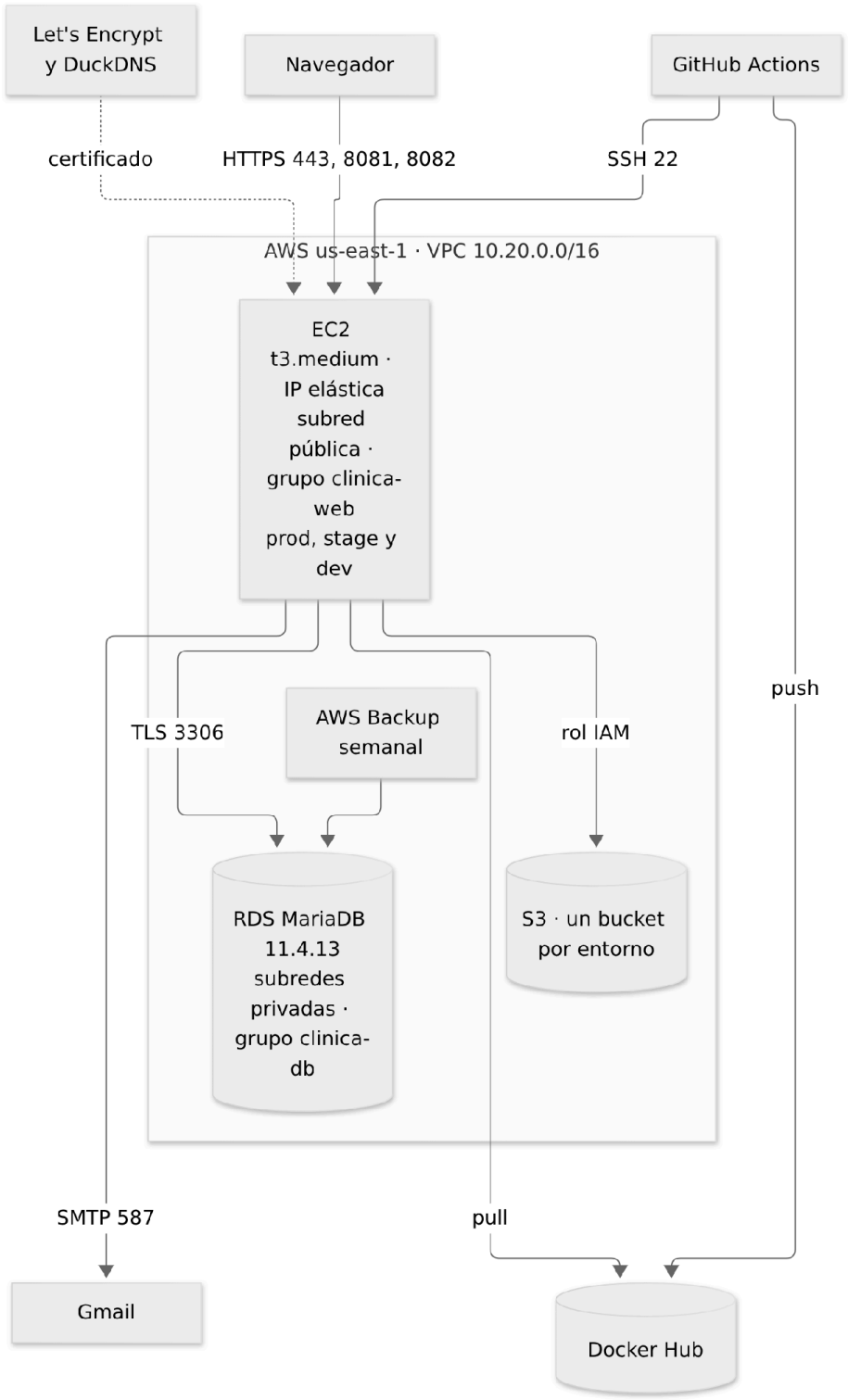
Guía de despliegue

El sistema corre en Amazon Web Services (AWS), región `us-east-1`. Terraform crea la infraestructura y GitHub Actions despliega la aplicación. Hay tres entornos, cada uno con su base de datos y su bucket:

Entorno	Dirección	Se despliega	Uso
prod	`https://pf-clinica.duckdns.org`	A mano desde `main`, con aprobación	La clínica
stage	`https://pf-clinica.duckdns.org:8081`	Solo, al fusionar un Pull Request en `main`	Validar la versión que irá a prod
dev	`https://pf-clinica.duckdns.org:8082`	Solo, con cada Pull Request abierto	Probar el cambio antes de fusionarlo

El costo es de unos USD 50 al mes. Un presupuesto de AWS avisa por correo al llegar al 80 % de USD 85.

Arquitectura desplegada



Recurso	Configuración
Red	VPC `10.20.0.0/16` con una subred pública y dos privadas en zonas distintas, sin NAT
Grupo `clinica-web`	Entrada por 22 (despliegue), 80 (redirección a HTTPS), 443, 8081 y 8082
Grupo `clinica-db`	Entrada por 3306 solo desde `clinica-web`
EC2	`t3.medium` (2 vCPU, 4 GB y 2 GB de *swap*), Ubuntu 24.04, disco de 30 GB cifrado, IMDSv2 obligatorio e IP elástica
RDS	MariaDB 11.4.13 `db.t4g.micro`, 20 GB cifrados, privada, TLS obligatorio, recuperación a un instante (PITR) de 7 días y protección contra borrado
S3	Un bucket por entorno (`clinica-<entorno>-archivos-<cuenta>`), privado y versionado
AWS Backup	Copia completa de RDS cada domingo a las 02:00 de Guatemala, guardada 4 semanas
IAM	Rol `clinica-ec2`: leer, escribir y borrar objetos solo en los tres buckets. Rol `clinica-infra`: lo asume únicamente el pipeline de infraestructura por OIDC. Rol de AWS Backup con la política administrada del servicio
Estado de Terraform	Bucket `clinica-tfstate-<cuenta>`, privado y versionado, con bloqueo por archivo
Presupuesto	USD 85 al mes con aviso al 80 %

Cada entorno es un proyecto de Docker Compose en la EC2, con estos contenedores:

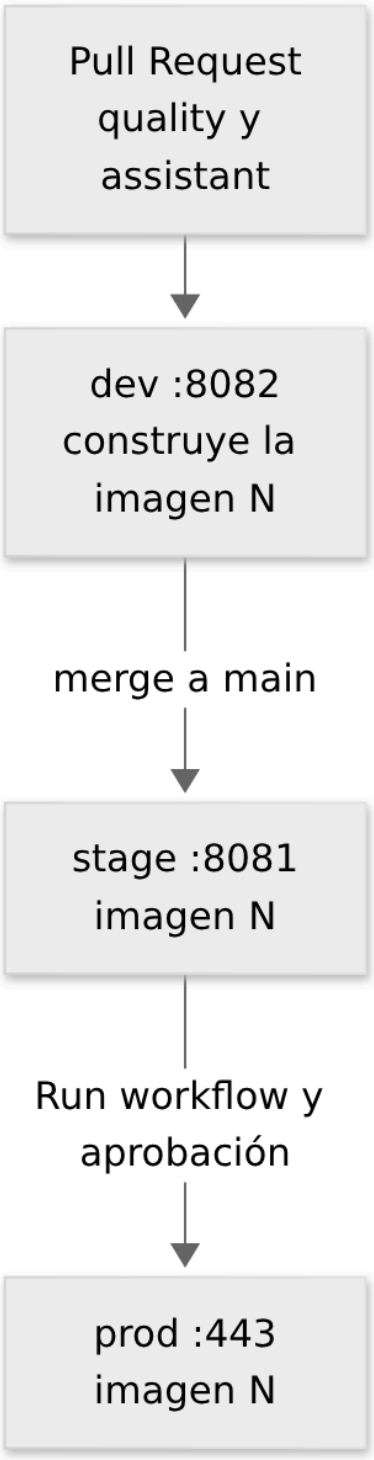
Contenedor	Qué hace
`web`	nginx 1.30.5 sin privilegios: termina TLS con HSTS, sirve los estáticos y pasa PHP a `app`
`app`	PHP-FPM 8.4 con Laravel 13, Filament 5 y Livewire 4
`worker`	Atiende las colas `critical`, `mail`, `documents` y `default`; se reinicia solo cada hora
`scheduler`	Concilia citas cada 5 minutos y vence pruebas psicométricas cada hora
`chatbot`	El asistente en Python 3.13 y FastAPI. No publica puertos: solo Laravel le habla, con token
`redis`	Solo caché. Perderlo no pierde datos: sesiones y colas viven en la base

Herramientas y versiones

Herramienta	Versión	Dónde
Terraform	1.16.6, en contenedor	Máquina del operador y pipeline `infra.yml`
Proveedor `hashicorp/aws`	6.69.1, fijado en `.terraform.lock.hcl`	`infra/`
Docker y Compose	29.9.1 y 5.6.0, instalados por `user-data.sh`	EC2

Herramienta	Versión	Dónde
GitHub Actions	Workflows `ci-cd.yml` e `infra.yml`	Repositorio `DiegoMaldonado19/clinica-app`
Registro de imágenes	Docker Hub: `clinica-app`, `clinica-web` y `clinica-chatbot`	Público; las imágenes no llevan secretos

Pipeline de integración y entrega continua



Job	Cuándo	Qué hace
`quality`	Cada Pull Request y cada push a `main`	Pint, PHPStan con Larastan, Deptrac, pruebas unitarias, migraciones sobre MariaDB, pruebas de integración y funcionales, una corrida sin caché, `composer audit` y gitleaks
`assistant`	Igual que `quality`	ruff, mypy, pytest con la batería de crisis y pip-audit del asistente
`deliver`	Si los dos anteriores pasan, o por despacho manual hacia prod	Construye las tres imágenes o reutiliza las del mismo árbol de archivos, las publica, entra por SSH, renueva TLS si hace falta, despliega y comprueba `/up` desde fuera

Una imagen por versión. Cada imagen lleva la huella del árbol de archivos. Si el árbol ya tiene imagen, se promueve esa misma: stage y prod reciben exactamente lo que se probó en dev. Prod nunca construye; toma la imagen del último despliegue sano de stage.

El despliegue en la EC2 lo hace `scripts/deploy.sh`:

1. crea la base y el usuario del entorno si no existen;
2. descarga las imágenes;
3. migra y siembra los catálogos con `php artisan migrate --force --seed`;
4. levanta los contenedores con `docker compose up --wait`;
5. borra las imágenes sin uso.

Las migraciones son versionadas y automáticas. Corren en cada despliegue y siempre hacia adelante: un cambio de esquema se corrige con una migración nueva, nunca editando una ya aplicada.

Los secretos viven en los Environments de GitHub (`dev`, `stage`, `prod` e `infra`). El pipeline escribe el `.env` de cada entorno en la EC2 y falla si falta un valor. Ninguno está en el repositorio, y gitleaks revisa cada Pull Request.

Flujo de ramas

El proyecto lo desarrolla una sola persona. Por eso usa desarrollo basado en tronco (*trunk-based*): una rama corta por cambio y un Pull Request hacia `main`. Así lo acordó

con los evaluadores desde la primera fase. Cada control de GitFlow tiene su equivalente:

Control	Cómo se cumple
Nada llega a `main` sin revisión automática	`main` exige Pull Request y los checks `quality` y `assistant` en verde, con la rama al día
Integración antes de producción	Cada Pull Request se despliega en dev; cada fusión, en stage
Producción solo con aprobación	El Environment `prod` exige la aprobación del responsable
Corrección urgente	Una rama corta desde `main` con su Pull Request: recorre dev y stage y se aprueba hacia prod
Infraestructura controlada	`infra.yml` corre solo a mano, desde `main` y con aprobación del Environment `infra`

Primer despliegue desde una cuenta vacía

Son unas dos horas, la mitad de espera.

#	Paso	Deja listo	Tiempo
1	Cuentas y llaves	AWS, SSH, DuckDNS, Gmail y Docker Hub	25 min
2	Verificaciones previas	Que el `apply` no falle a medias	5 min
3	Bootstrap	Bucket del estado y rol OIDC de GitHub	3 min
4	Infraestructura	VPC, EC2, RDS, S3, respaldos y presupuesto	20 min
5	GitHub	Variables, Environments, secretos y protección de `main`	20 min
6	dev, stage y prod	Los tres entornos con la misma imagen	35 min
7	Primer ingreso	La psicóloga con contraseña y segundo factor	5 min
8	Infraestructura por el pipeline	Un `plan` sin cambios por OIDC	5 min

1. Cuentas y llaves

Cada valor se guarda en un gestor de contraseñas: GitHub no deja volver a leer un secreto.

Dato	Cómo se obtiene
Llave de AWS	Usuario IAM `clinica-admin` con `AdministratorAccess`, solo para el bootstrap. Se desactiva al terminar
Llave SSH del pipeline	`ssh-keygen -t ed25519 -f ~/.ssh/clinica-ec2 -N '' -C clinica-pipeline`
Subdominio y token de DuckDNS	En duckdns.org. La IP la registra la EC2
Contraseña de aplicación de Gmail	Con la verificación en dos pasos de la cuenta activa; se copia sin espacios

Dato	Cómo se obtiene
Token de Docker Hub	De lectura y escritura
Datos de la clínica	Teléfono, correo, dirección y línea de crisis confirmada, que la landing muestra siempre

2. Verificaciones previas

- La cuenta de AWS debe estar en el plan de pago: el plan gratuito rechaza la `t3.medium`.
- La AMI de Ubuntu fijada en `ec2.tf` debe existir en la región.
- Si la cuenta ya tiene el proveedor OIDC de GitHub, se importa antes de aplicar el bootstrap.

Terraform corre en contenedor, con la misma versión que el pipeline:

```
alias terraform='docker run --rm -it -u "$(id -u):$(id -g)" -e HOME=/tmp -v
"$HOME/.aws:/tmp/.aws:ro" \
  -e AWS_PROFILE -e TF_VAR_db_admin_password -e TF_VAR_ssh_public_key -e
TF_VAR_alert_email \
  -v "$PWD:/w" -w /w hashicorp/terraform:1.16.6'
```

3. Bootstrap, una sola vez

```
cd infra/bootstrap
terraform init && terraform apply      # anota state_bucket e infra_role_arn
```

4. Infraestructura

```
cd ..
export TF_VAR_db_admin_password="$(openssl rand -hex 20)"   # es
DB_ADMIN_PASSWORD
export TF_VAR_ssh_public_key="$(cat ~/.ssh/clinica-ec2.pub)"
export TF_VAR_alert_email="correo-del-responsable"
terraform init -backend-config="bucket=<state_bucket>"
terraform apply                         # anota ec2_host y db_host; RDS tarda
unos 15 minutos
```

La contraseña maestra de RDS es de solo escritura: no queda en el estado de Terraform.

5. GitHub

Variables del repositorio: `DOMAIN`, `CLINIC_PHONE`, `CLINIC_EMAIL`, `CLINIC_ADDRESS` y `CLINIC_CRISIS_LINE`. Son datos públicos y van como variables, no como secretos.

Environments:

Environment	Ramas permitidas	Aprobación
`dev`	Cualquiera (corre sobre el Pull Request)	No
`stage`	`main`	No
`prod`	`main`	Sí, del responsable
`infra`	`main`	Sí, del responsable

Secretos de dev, stage y prod. «Igual» lleva el mismo valor en los tres entornos y «distinto» se genera aparte para cada uno.

Secreto	Valor
`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`	Igual: la cuenta de Docker Hub
`EC2_HOST`, `EC2_SSH_KEY`	Igual: la salida `ec2_host` y la llave privada del pipeline
`DB_HOST`, `DB_ADMIN_PASSWORD`	Igual: la salida `db_host` y la contraseña maestra
`DB_PASSWORD`	Distinto: `openssl rand -hex 24`
`APP_KEY`, `CLINICAL_ENCRYPTION_KEY`	Distintos entre sí y por entorno: `base64:` más `openssl rand -base64 32`. Perder la llave clínica de prod es perder las notas
`CHATBOT_API_TOKEN`	Distinto: `openssl rand -hex 32`
`AWS_BUCKET`	Distinto: `clinica-<entorno>-archivos-<cuenta>`
`MAIL_USERNAME`, `MAIL_PASSWORD`	Igual: la cuenta de Gmail, que también es el remitente, y su contraseña de aplicación
`ADMIN_EMAIL`	Igual: el correo real de la psicóloga, con el que entra la primera vez
`DUCKDNS_TOKEN`, `TLS_EMAIL`	Igual: el token de DuckDNS y el correo de avisos de Let's Encrypt
`ANTHROPIC_API_KEY`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`	Opcionales. Sin la primera, el asistente se despliega apagado; sin las otras dos, todo aviso sale por correo

Secretos de `infra`: `AWS_ROLE_ARN` (la salida `infra_role_arn`), `TF_STATE_BUCKET`, `TF_VAR_db_admin_password`, `TF_VAR_ssh_public_key` y `TF_VAR_alert_email`.

Protección de `main`: Pull Request obligatorio sin aprobaciones (el único desarrollador no puede aprobarse a sí mismo), checks `quality` y `assistant` obligatorios y rama al día.

6. dev, stage y prod

1. **dev:** se abre el Pull Request. `deliver` construye las imágenes y despliega. Debe responder `https://<dominio>:8082/up`.

2. **stage:** al fusionar, se despliega la misma imagen en `https://<dominio>:8081`.
3. **prod:** Actions → CI/CD → *Run workflow* desde `main` → *Review deployments* → *Approve*.

El primer despliegue de cada entorno espera a que la EC2 termine de instalar Docker, emite el certificado de Let's Encrypt (uno para los tres puertos), crea la base y siembra los catálogos y la clínica.

7. Primer ingreso

La psicóloga entra a `https://<dominio>/admin` por «¿Olvidaste tu contraseña?» con el correo de `ADMIN_EMAIL`. Elige su contraseña, configura el segundo factor, guarda sus códigos de recuperación y crea desde el panel el usuario de recepción.

8. Infraestructura por el pipeline

Actions → Infraestructura → *Run workflow* → `plan` → aprobación. Debe terminar en `No changes`: prueba que GitHub entra a AWS por OIDC, sin llaves guardadas. Después se desactiva la llave de `clinica-admin`.

Despliegue de una versión nueva

1. Abrir el Pull Request y esperar `quality`, `assistant` y el despliegue en dev.
2. Probar el cambio en `https://<dominio>:8082`.
3. Fusionar. Esperar el despliegue en stage y probar en `https://<dominio>:8081`.
4. Actions → CI/CD → *Run workflow* desde `main` y aprobar `prod`.
5. Verificar prod con la lista de abajo.

Verificación después de desplegar

Comprobación	Cómo	Resultado esperado
Salud	`curl -s -o /dev/null -w '%{http_code}' https://<dominio>/up`	`200` en los tres puertos
Redirección	`curl -sI http://<dominio>/`	`301` hacia `https://`
Certificado	`openssl s_client -connect <dominio>:443 -servername`	Emitido por Let's Encrypt y vigente

Comprobación	Cómo	Resultado esperado
	`<dominio> </dev/null | openssl x509 -noout -issuer -dates`	
Imagen	En la EC2: `cd ~/clinica/prod && docker compose ps`	El mismo número de imagen que stage, contenedores sanos
Ingreso	`https://<dominio>/admin` y `/portal`	La pantalla de ingreso, sin errores

Si algo falla

Síntoma	Causa probable	Qué hacer
«Escribir el .env» falla por valores faltantes	Una variable guardada como secreto, o un secreto vacío	Mover `DOMAIN` y `CLINIC_*` a variables y completar lo que indica el registro
`Could not assume role with OIDC`	`AWS_ROLE_ARN` distinto del rol creado, o la confianza no coincide con el repositorio	Comparar con `terraform output -raw infra_role_arn` en `infra/bootstrap`
`docker/login-action` falla	Token de Docker Hub vencido	Crear otro de lectura y escritura
No conecta por SSH	`EC2_HOST` equivocado, o la EC2 aún arranca	Revisar `terraform output ec2_host` y repetir
`DuckDNS rechazo la actualizacion: KO`	Token o subdominio	Copiar de nuevo el token; `DOMAIN` es `<subdominio>.duckdns.org`
«Verificar desde fuera» falla	Un contenedor no quedó sano	`docker compose ps` y `docker compose logs app web` en la EC2
prod: `Stage todavia no tiene un despliegue exitoso`	stage no terminó bien	Desplegar stage primero

Límites declarados

- Los tres entornos comparten la EC2 y su rol IAM; aislarlos exige una instancia por entorno.
- El puerto 22 está abierto porque los ejecutores de GitHub no tienen IP fija. Solo se entra con la llave del pipeline.
- No hay vuelta atrás automática. Si los contenedores nuevos no quedan sanos, el entorno sigue con esa versión hasta desplegar la anterior. Si falla antes, al migrar, siguen sirviendo los contenedores anteriores.

Trazabilidad

Historial de entregas

Fase	Fechas	Qué se entregó
1 · Análisis	Agosto de 2026	Contexto, actores, procesos actuales y objetivo en BPMN, 13 indicadores, análisis económico y de riesgos, requisitos, 35 casos de uso, arquitectura inicial y el backlog en Jira
2 · Diseño	Agosto y septiembre	Stack y decisiones 001 a 022, arquitectura hexagonal, nube, CI/CD, seguridad, modelo de datos, 26 historias con criterios Gherkin, clases, secuencias y experiencia de usuario
2 · Construcción	8 al 20 de septiembre	El sistema local con su CI (incrementos F-01 a F-12)
3 · Despliegue y cierre	17 al 25 de octubre	Infraestructura y despliegue, seguridad avanzada, reportes, asistente, pruebas psicométricas, endurecimiento y mejoras de uso (incrementos F3-01 a F3-10)

Del enunciado al sistema

Exigencia	Cómo se cumple	Evidencia
Autenticación con registro, sesiones seguras y recuperación	Cuenta creada al agendar o por recepción; sesiones en la base; recuperación por enlace de un solo uso	`AuthTest`, `LoginTest`
Segundo factor por correo	Código de 6 dígitos que vence en 10 minutos, o aplicación TOTP	`TwoFactorTest`
Al menos dos roles	Psicóloga (administradora), recepción y paciente, más roles creados por la administradora	`AbilityTest`, `PanelTest`
CRUD completo de una entidad principal	Citas, pacientes, servicios, usuarios, bloqueos y preguntas frecuentes	`AdminPanelTest`, `PortalTest`
Módulo de comunicación	25 avisos por correo con ventana de silencio y preferencias, y el asistente conversacional	`NotificationTest`, `ChatWidgetTest`
Métricas del administrador con exportación PDF y Excel	Tablero de 13 indicadores con las dos exportaciones	`ReportTest`, `ReportServiceTest`
Despliegue en la nube con HTTPS	AWS con TLS de Let's Encrypt y redirección de HTTP	`https://pf-clinica.duckdns.org`
CI/CD e infraestructura como código	GitHub Actions y Terraform en el repositorio	`ci-cd.yml`, `infra.yml`, `infra/`
Migraciones versionadas y automáticas	18 migraciones aplicadas por el pipeline	`deploy.sh`

Exigencia	Cómo se cumple	Evidencia
Respaldo completo, incremental y de transacciones	AWS Backup semanal, instantánea diaria y registro binario cada 5 minutos	Terraform `s3.tf` y `rds.tf`
Gestión de secretos	Environments de GitHub; gitleaks	`ci-cd.yml`

Requisitos funcionales

Área	Requisitos	Construido en	Completado en la fase 3
Agendamiento público	RF-01 a RF-09	F-05, F-06	El asistente agenda por la API
Aprobación y SLA	RF-10 a RF-15	F-07	—
Pagos	RF-16 a RF-23	F-08	Comprobante dentro del panel y crédito a favor
Cancelación, inasistencia y recordatorios	RF-24 a RF-31	F-09, F-11	Reprogramar en un paso; inasistencia automática al terminar la sesión
Atención clínica y expediente	RF-32 a RF-41	F-10, F-11	Pruebas psicométricas
Reportes y administración	RF-42 a RF-48	F-11	Tablero y exportación
Acceso y seguridad	RF-49 a RF-53	F-03	Segundo factor y contraseñas de 12; roles del personal; fotos con permiso
Notificaciones	RF-54 a RF-58	F-04	Preferencias por canal y WhatsApp listo
Asistente conversacional	RF-59 a RF-63	—	Construido en la fase 3; se despliega apagado sin llave

Fuera de alcance	Por qué	Cuándo
RF-64 WhatsApp	Exige la verificación del negocio por Meta	Canal listo; se activa con dos secretos
RF-65 Pasarela de pago	Exige contrato con un adquirente	Puerto listo
RF-66 Facturación FEL	Exige un certificador autorizado	Puerto listo
RF-67 Liquidación a varios terapeutas	Hay una sola terapeuta	Al contratar otra
RF-68 Multicliente	Hay una sola clínica	Al firmar la segunda
RF-69 Videoconsulta	No es una necesidad declarada	Sin fecha
RF-70 Aplicación móvil nativa	La landing y el portal son adaptables al móvil	Sin fecha
RF-71 Recuperación vectorial	El conocimiento cabe en la instrucción del modelo	Si crece

Historias de usuario, código y pruebas

HU	Historia	Construida en	Pruebas
HU-01	Consultar disponibilidad real	`AvailabilityCalculator`, `BookingWizard`	`BookingWizardTest`, `AvailabilityCalculatorTest`
HU-02	Solicitar una cita como paciente nuevo	`BookingService::request()`	`BookingTest`, `BookingWizardTest`
HU-03	Reconocer a un paciente que regresa	`DatabasePatientRegistry::findOrRegister()`	`BookingTest`
HU-04	Bandeja de aprobaciones por SLA	`ApprovalInbox`	`AdminPanelTest`, `SlaTest`
HU-05	Expiración automática por SLA	`ExpirePreAppointmentJob`, `appointments:reconcile`	`SlaTest`, `AppointmentTest`
HU-06	Subir comprobante de transferencia	Portal «Registrar mi pago»	`PaymentTest`, `PortalTest`
HU-07	Conciliar pagos	`PaymentResource`	`PaymentTest`
HU-08	Cancelación automática por impago	`CancelUnpaidJob`	`CancellationTest`
HU-09	Cancelar con la política a la vista	`TieredCancellationPolicy`, `BookingService::reschedule()`	`CancellationTest`, `RescheduleTest`, `TieredCancellationPolicyTest`
HU-10	Recordatorios automáticos	`SendReminderJob`	`CancellationTest`, `NotificationTest`
HU-11	Registrar nota SOAP	`SessionNote`, `ClinicalNote`	`ClinicalRecordTest`
HU-12	Restricción clínica para recepción	Políticas y habilidades	`ClinicalRecordTest`, `AbilityTest`
HU-13	Asignar y responder pruebas	`PsychTestService`, `MyTests`	`PsychTestTest`
HU-14	Tablero de indicadores	`Dashboard`, `ReportService`	`ReportTest`, `ReportServiceTest`
HU-15	Exportar a Excel y PDF	`ReportFileExporter`	`ReportTest`
HU-16	Restricción de reportes financieros	`report.view.financial`	`ReportTest`, `AbilityTest`
HU-17	Configurar reglas de negocio	`BusinessRules` (panel «Plazos y políticas»)	`AdminPanelTest`, `PanelTest`
HU-18	Bitácora de acceso al expediente	`AuditLog`, `AuditLogEntryResource`	`ClinicalRecordTest`, `AdminPanelTest`
HU-19	Registrar inasistencia	`AppointmentTransitions`, conciliación	`CancellationTest`, `AppointmentTest`
HU-20	Bloquear la agenda	`ScheduleBlockResource`	`AdminPanelTest`, `RescheduleTest`

HU	Historia	Construida en	Pruebas
HU-21	Segundo factor por correo	`EmailCodeAuthentication`	`TwoFactorTest`, `LoginTest`
HU-22	Cambio obligatorio de contraseña temporal	`EnforceTemporaryPassword`	`AuthTest`, `AccountTest`
HU-23	Recuperar mi contraseña	`RequestPasswordReset`	`AuthTest`
HU-24	Resolver dudas por chat	`chatbot/app/assistant.py`, `ChatWidget`	`ChatWidgetTest`, `ChatbotApiTest`, `test_assistant.py`
HU-25	Agendar desde el chat	`POST /api/v1/appointments`	`ChatWidgetTest`, `ChatbotApiTest`
HU-26	Escalar señales de crisis	`chatbot/app/crisis.py`	`test_crisis_escalation.py` (20 casos)

Procesos y su implementación

Proceso	Dónde vive
Agendar desde la web o el asistente	`BookingService::request()`, `BookingWizard`, `POST /api/v1/appointments`
Aprobar, rechazar y expirar por SLA	`AppointmentTransitions`, `ExpirePreAppointmentJob`
Pagar, conciliar y cancelar por impago	`BillingService`, `CancelUnpaidJob`
Cancelar y reprogramar con cargo por tramos	`BookingService::reschedule()`, `TieredCancellationPolicy`, `CreditAllocation`
Atender y sellar la nota	`SessionNote`, `ClinicalNote`
Notificar con ventana de silencio e idempotencia	`NotificationDispatcher`, `SendNotificationJob`
Reportes	`ReportService`, `ReportFileExporter`
Bloquear la agenda	`ScheduleBlockResource`
Pruebas psicométricas	`PsychTestService`, `psych-tests:expire`
Conversar, agendar y escalar una crisis	`chatbot/app`, `ChatWidget`

Verificación al cierre

Verificación	Resultado al 25 de octubre de 2026
Pruebas de PHP	318 en verde, más 199 en la corrida sin caché
Pruebas del asistente	En verde, con la batería de crisis al 100 %
Arquitectura	Deptrac sin violaciones ni dependencias sin clasificar
Calidad	Pint y PHPStan sin errores
Dependencias	`composer audit` y pip-audit sin avisos

Verificación	Resultado al 25 de octubre de 2026
Despliegue	Los tres entornos con la misma imagen; `/up` en 200; TLS válido hasta el 23 de enero de 2027